Full but Not Flooded: an AI Assistant Synthesises a Storm Water Pond Controller with Uppaal Stratego

Author Names

Institution

Abstract. A user asked an AI assistant, connected to Uppaal through a tool interface, to build a Uppaal Stratego model of an urban storm water pond and to learn a periodic controller for its outlet that avoids overflow while keeping the water level as high as possible. In one session the assistant built a stochastic hybrid model with four rain scenarios, learned and evaluated a controller, and explained the model. The session shows what changes when the task is synthesis rather than verification: a timid controller pointed to a mis-calibrated plant, safety became a probability with an error bar, the learned strategy turned out to live outside the model file, and the learning time was dominated by a simulation setting rather than by learning parameters. We summarise the model and results and reflect on what the session suggests for AI-assisted controller synthesis.

1 Setting

The user asked for "a UPPAAL Stratego model of a storm water pond" that collects rain water from a neighbouring urban area, under different rain scenarios, together with a periodic controller that sets an outlet "to different degrees" so that the level stays between a minimum and a maximum, balancing safety (no overflow) against optimality (a high average level). The assistant had direct access to the open Uppaal [4] editor: it could read and replace the model, edit declarations, locations and queries, check syntax and run queries. Later in the session it also used the command-line engine `verifyta`, which it found installed on the machine. The user asked questions and took the decisions about scope and next steps.

Compared with timed-automata verification sessions, this case differs in three ways: the plant is *hybrid* (a water balance given by an ODE), the environment is *stochastic* (rain), and the goal is *synthesis*: a strategy is learned [1,2] and then evaluated by statistical model checking, so every answer is an estimate with a confidence interval. Storm water ponds are an established Uppaal Stratego case [3]; the session rebuilt one from a one-paragraph request.

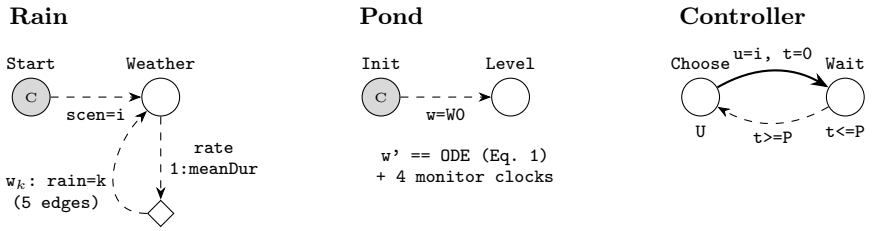


Fig. 1. The three templates. Grey locations (C) are committed, `U` is urgent, the diamond is a probabilistic branch point. Solid edges are controllable, dashed edges uncontrollable. The controller's edge selects $\texttt{i} \in \{0, \ldots, 3\}$.

Table 1. Long-run share of time per rain class and mean rainfall implied by the rain tables (computed by the assistant during the session).

Scenario	% time dry / light / mod. / heavy / extreme	rain/day
dry summer	94.8 / 4.3 / 0.8 / 0.2 / 0.0	4.5 mm
normal	73.7 / 18.4 / 6.0 / 1.7 / 0.2	34 mm
wet autumn	23.0 / 46.0 / 25.3 / 5.2 / 0.4	108 mm
cloudburst	91.4 / 2.9 / 2.5 / 2.2 / 1.1	35 mm

2 The Model

Time is in minutes and levels in metres. Three templates (Fig. 1) interact only through shared variables: the rain class `rain`, the scenario `scen` and the outlet setting `u`.

Rain. Rain is a continuous-time Markov chain over five classes (dry, light, moderate, heavy, extreme: 0, 2, 8, 25, 60 mm/h). At start a scenario is drawn uniformly (or fixed by a constant): dry summer, normal, wet autumn, or cloudburst. In location `Weather` the sojourn time is exponential with mean `meanDur[scen][rain]`; at the branch point the next class is drawn with weights from a table with one row for starting a rain event and one for continuing or ending it. Table 1 shows the long-run behaviour that the assistant derived from the tables when the user asked for an explanation.

Pond. The level w is a hybrid clock whose rate is a water balance over a pond of area $A = 5000\,\mathrm{m}^2$:

$$\dot{w} = \frac{1}{A}\Big(Q_{\mathrm{in}}(\mathit{rain}) - \frac{u}{3}\,Q_{\max}\sqrt{w/w_{\max}} - C_s\,(w - w_{\max})_+^{3/2}\Big) - e\,[w > 0] \quad (1)$$

Inflow is rain intensity times an effective catchment of 18 ha; the outlet follows an orifice law; above $w_{\max} = 2.5\,\mathrm{m}$ an emergency spillway follows a weir law, so overflow is part of the physics rather than an error location; e is evaporation and infiltration. The minimum level is 1.0 m. Four further hybrid clocks act as

Table 2. Results over a 24 h horizon with a random scenario (1000 runs, 95 % intervals). $Q_{\max}$ is the fully open outlet flow at $w_{\max}$.

	$Q_{\max} = 60\,\mathrm{m}^3/\mathrm{min}$		$Q_{\max} = 120\,\mathrm{m}^3/\mathrm{min}$	
	random	`Opt`	random	`Opt`
average level	0.47 m	1.08 m	0.22 m	**1.32 m**
P(overflow)	6.4 ± 4.0 %	3.6 ± 3.4 %	≤ 5 %	5.5 ± 3.9 %
overflow severity	–	0.30 m·min	–	0.06 m·min
P(below minimum)	–	94 %	–	58 %

monitors: $\texttt{lvlInt}' = w$ (average level), $\texttt{dist}' = w_{\max} - w$, $\texttt{excess}' = (w - w_{\max})_+$ and $\texttt{deficit}' = (w_{\min} - w)_+$.

Controller. Every $P = 30$ minutes the controller chooses $u \in \{0, 1, 2, 3\}$ (closed, $\frac{1}{3}$, $\frac{2}{3}$, fully open). The initial location is urgent rather than committed so that the committed initialisations of `Rain` and `Pond` happen before the first choice.

3 Learning a Periodic Controller

The strategy is learned by minimising one cost, with safety as a heavily weighted penalty:

```
strategy Opt = minE (dist + PEN_OVER*excess + PEN_UNDER*deficit) [<=H]
              { rain } -> { w } : <> time >= H  // H=1440, PEN_OVER=10000, PEN_UNDER=100
E[<=H; 1000] (max: lvlInt / H) under Opt              // average level
Pr[<=H] (<> excess > 0.0) under Opt                    // overflow probability
```

The controller observes the rain class and the level. Because `excess` only grows, `excess > 0` at the end of a run records an overflow even if it happened between two simulation steps, and its value grades how bad it was. Table 2 summarises the results.

4 What the Session Revealed

A timid controller pointed to the plant. With the first outlet capacity the learned strategy kept the pond at the minimum (average 1.08 m, below the minimum in 94 % of the runs). Rather than tuning the learner, the assistant traced this to the plant: extreme rain delivers $180\,\mathrm{m}^3/\mathrm{min}$, the fully open outlet only 60, so long cloudbursts overflow whatever the controller does, and the overflow penalty pushes the strategy to the lowest level it can hold. Doubling the outlet to $2\,\mathrm{m}^3/\mathrm{s}$ raised the average level by about 0.25 m and cut overflow severity five-fold. Steady-state reasoning made the control problem concrete: a fully open outlet holds heavy rain at 0.98 m, $\frac{2}{3}$ open at 2.2 m, and nothing holds extreme rain, so the controller must keep empty volume in reserve. In synthesis a poor strategy can be a correct answer to a badly posed plant.

Safe by being empty. The random baseline overflows about as rarely as the learned controller, but only because it keeps the pond almost empty (0.22 m). Safety and optimality can only be judged together, which is exactly what the single cost expresses.

Safety is soft. Safety enters only as a penalty, and rain durations are exponential, hence unbounded: some overflow risk always remains, and with the default precision the estimate ($5.5 \pm 3.9\,\%$) cannot separate the controllers. The assistant named the route to a guarantee: bounded rain durations and a safety shield computed on a discretised abstraction, with optimisation under the shield.

The assistant flagged its own assumptions. When explaining the rain model the assistant computed Table 1 and pointed out that 34 and 108 mm/day are far above a Danish average of about 2 mm/day: the scenarios are stress tests, not climatology. All parameters were plausible defaults chosen by the assistant, and the results are conditional on them.

5 Where the Tools Shaped the Work

Keywords. Two syntax errors came from names that are UPPAAL keywords: `trans`, used for the transition table, and `under`, used for a cost clock, which collides with `... under Opt`. The assistant renamed them to `nextRain`, `excess` and `deficit`.

Where does a strategy live? After re-running the evaluation the user found the learned strategy gone and asked why. Strategies exist only in the engine's memory: they are discarded when an edited model is re-sent to the engine or the engine restarts, and they are never written to the model file. The assistant added `saveStrategy` and `loadStrategy` queries and checked the round trip (average level 1.31 m after reloading). A loaded strategy is only valid for an unchanged model, which makes the strategy file an artefact to be versioned with the model.

"10 minutes is a long time to wait for an answer." Learning took about 7 minutes in the GUI and blocked the tool interface, whose calls timed out while the engine reported itself busy. The user asked whether the learning parameters could be set. The tool interface could not set engine options, so the assistant ran `verifyta` directly and measured (Table 3). The cost was not in learning but in simulating the ODE: the default discretisation step of 0.01 min is far finer than dynamics that evolve over tens of minutes. 1000 plain simulations took 55 s at step 0.01, 7 s at 0.5 and 3.5 s at 1.0, with the same average level within the confidence interval. Shrinking the learning budget instead was fast but produced a strategy that kept the pond high with about 30 times the overflow severity: an undertrained strategy fails on the rare storms that the penalty is meant to exclude, and only an independent evaluation reveals it.

Table 3. Learning experiments with `verifyta` (one seed per configuration; total time includes 3×1000 evaluation runs). Objective: $dist + 10^4 \cdot excess + 100 \cdot deficit$, lower is better. *Slowed by running in parallel with the others.

	step	budget	time	avg level	excess	deficit	objective
A	0.01	default	16.5 min*	1.16 m	0.05 ± 0.04	5.2	2 900
B	0.5	default	2.1 min	1.38 m	0.09 ± 0.06	0.8	2 600
C	1.0	default	1.2 min	1.20 m	0.04 ± 0.03	2.0	2 500
D	0.5	small	0.6 min	1.61 m	2.5 ± 0.5	0.3	26 700

Protecting the user's work. The editor still held an unrelated, unsaved model from an earlier session. Before replacing it the assistant saved a copy to disk; replacing a model through the tool interface leaves no undo history.

6 Reflections

Synthesis moves the questions. In the timed-automata verification sessions we compare with, the user's questions concern what a property means; here they concerned the plant (is the outlet large enough?), the artefacts (where is my strategy?) and the cost of an answer (why ten minutes?). Each was answered with a measurement rather than an assurance: a steady-state calculation, a save–load round trip, a timing table.

Evaluation is part of the method. A learned strategy is a hypothesis. The session kept learning and evaluation separate, compared against a random baseline, and reported intervals; this is what exposed both the mis-calibrated plant and the under-trained strategy.

Explanation built on numbers. The explanations the user asked for (the rain model, the pond template, which query gives the average level) worked best when grounded in derived quantities: long-run rain shares, steady-state levels, drain times.

The tool interface shapes the work. Synchronous calls with a timeout fit verification queries of seconds but not learning of minutes; engine options were out of reach of the interface but not of the assistant, which used the command line. A non-blocking learning call and access to engine options would remove both obstacles.

Outlook. The open points are standard: a rain forecast as additional observation, a stage–area curve and runoff delay in the plant, a shield for hard safety, and rain statistics calibrated on gauge data. This summary was written by the assistant from the session record.

References

1. David, A., Jensen, P.G., Larsen, K.G., Mikučionis, M., Taankvist, J.H.: Uppaal Stratego. In: TACAS 2015. LNCS, vol. 9035, pp. 206–211. Springer (2015)

2. Jaeger, M., Jensen, P.G., Larsen, K.G., Legay, A., Sedwards, S., Taankvist, J.H.: Teaching Stratego to play ball: optimal synthesis for continuous space MDPs. In: ATVA 2019. LNCS, vol. 11781, pp. 81–97. Springer (2019)
3. Goorden, M.A., Larsen, K.G., Nielsen, J.E., Nielsen, T.D., Rasmussen, M.R., Srba, J.: Learning safe and optimal control strategies for storm water detention ponds. IFAC-PapersOnLine **54**(5) (ADHS 2021) (2021)
4. Behrmann, G., David, A., Larsen, K.G.: A tutorial on Uppaal. In: SFM-RT 2004. LNCS, vol. 3185, pp. 200–236. Springer (2004)